Amazon OpenSearch Service

Explore Amazon OpenSearch Service, a managed AWS tool for indexing, searching, and visualizing large datasets. Learn about its architecture, ingestion methods, querying capabilities, and security features. Understand how to integrate OpenSearch with AWS services for real-time streaming or batch analytics, optimize performance, and troubleshoot common issues.

OpenSearch is a managed service based on open-source Elasticsearch and Kibana. It provides fast access to large amounts of data for analysis and offers an OpenSearch dashboard for visualization and exploration. It supports multiple analytical capabilities, such as k-nearest neighbors (KNN) search, SQL, anomaly detection, machine learning commons, trace analytics, full-text search, and more.

OpenSearch consists of clusters, which are made up of nodes, each of which stores data in indexes. These indexes are divided into shards, distributing data across nodes for fault tolerance and scalability.

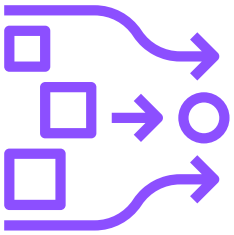
The core components of OpenSearch include:

- **OpenSearch domain:** An OpenSearch domain is essentially an ElasticSearch cluster. It is a collection of nodes and configurations that define an OpenSearch service environment. EC2 instances act as nodes in

Ask

this cluster, which allows us to process large volumes, execute complex queries, and perform aggregations.

- **Data Prepper:** OpenSearch Ingestion is a fully managed, serverless data collector that enables real-time ingestion of logs, metrics, and traces without third-party tools like LogStash. Powered by the open-source Data Prepper, it supports real-time streaming, batch processing, and data transformation with features like buffering, schema validation, and error handling. It benefits from automatic scaling, cost control, enhanced security with VPC integration, and regular updates.



- **OpenSearch dashboards:** An open-source visualization tool provided with each OpenSearch domain. It is a JavaScript application that provides functionality similar to the Kibana dashboards in the ElasticSearch clusters.

Indexing and queries

When we send JSON-formatted documents to OpenSearch, it automatically indexes the data upon ingestion. This means that OpenSearch analyzes and stores the data to make it searchable, filterable, and aggregatable, without requiring manual indexing steps.

Ask

Suppose we require more control over how the data is indexed, for instance, specifying how certain fields are analyzed or stored. In that case, we can define custom mappings and index settings before data ingestion. Otherwise, OpenSearch will infer mappings dynamically based on the structure of the first ingested documents.

An **index** in OpenSearch is similar to a database table, and each document is a unit of searchable data, like a log entry, a product, or a telemetry record.

Once our data is indexed, OpenSearch offers two primary methods for querying it: URI-based queries and request body queries using the Query DSL.

- URI queries are concise and suitable for simple lookups, passed as parameters directly in the URL.
- Domain-specific language (DSL) queries, on the other hand, provide far greater flexibility and are sent in the request body. This method supports complex operations like filtering, nested queries, and aggregations, all essential for advanced analytics and dashboard applications. For example, logs, products, or telemetry entries are all typical document types.

Here are sample formats for both methods:

URI

Ask

```
<METHOD> <opensearch-domain>/<index>/<operation>?q=field:value
```

URI format for querying OpenSearch

Example: Monitoring solution for IoT devices

Let's consider a scenario where we are developing a monitoring solution for IoT devices that report their current temperature readings and location. Each IoT device sends telemetry in the following JSON format:

index.js Node.js

```
{
  "device_id": "sensor-1024",
  "location": "warehouse",
  "temperature": 26.5,
  "timestamp": "2025-06-18T12:00:00Z"
}
```

Ask

Indexed IOT data

Data indexing using `POST`

To ingest this data into OpenSearch, we use a `POST` request to the `iot-devices` index as follows:

main.sh Shell

```shell
POST <opensearch-domain>/iot-devices/_doc
{
  "device_id": "sensor-1024",
  "location": "warehouse",
  "temperature": 26.5,
  "timestamp": "2025-06-18T12:00:00Z"
}
```

Add telemetery data from sensors

OpenSearch will automatically assign a unique ID to each document so multiple readings from the same device can be stored as separate entries.

This document is added to the `iot-devices` index. This approach allows each telemetry reading, regardless of device, to be indexed independently, preserving historical data and enabling time-series analysis. Over time, this index will store thousands of similar documents from various sensors, allowing us to run complex queries or statistical aggregations.

Data indexing using `PUT`

Ask

We can also use `PUT` request to add data to OpenSearch. It is used when we want to control the document ID ourselves. If the specified ID already exists in the

index, the document will be replaced. This makes `PUT` ideal when documents need to be updated or uniquely tracked over time. Here's what the query to add data using the `PUT` query looks like:

main.sh Shell

```shell
PUT <opensearch-domain>/iot-devices/_doc/sensor-1024
{
  "device_id": "sensor-1024",
  "location": "warehouse",
  "temperature": 26.5,
  "timestamp": "2025-06-18T12:00:00Z"
}
```

Using PUT method to add data to OpenSearch

Note: `POST` is used when we want OpenSearch to automatically assign a unique ID to each new document. This is useful when every document should be stored independently, common for log entries or telemetry data. `PUT`, on the other hand, is used when we want to control the document ID ourselves.

Batch indexing

For batch ingestion, a `_bulk` API call is often used to send multiple documents in a single request. as follows:

Ask

main.sh Shell

```shell
POST <opensearch-domain>/index-name/_bulk
```

```
{ "index": {"_id": "1"} }
{ "your": "first document" }
{ "index": {"_id": "2"} }
{ "your": "second document" }
```

Add bulk data to Opensearch

Data indexing through a Lambda function

We can also use a Lambda function for the ingestion of the data to OpenSearch as follows:

main.py Python

```
import json
import requests
from requests_aws4auth import AWS4Auth
import boto3

region = 'us-east-1'
service = 'es'
credentials = boto3.Session().get_credentials()
auth = AWS4Auth(credentials.access_key, credentials.secret_key, regio

opensearch_url = 'https://your-domain.us-east-1.es.amazonaws.com'
index = 'iot-devices'
```

Ask

Lambda function code to add data to OpenSearch

Here's a brief, line-by-line explanation of the provided Python code:

- **Lines 1–4:** Imports necessary libraries: `json` for data handling, `requests` for HTTP requests, `requests_aws4auth` for AWS authentication, and `boto3` for AWS SDK interactions.
- **Lines 6–9:** Sets up AWS region and service for OpenSearch, then retrieves current AWS credentials and creates an `AWS4Auth` object for signing requests.
- **Lines 11–12:** Define the target OpenSearch domain URL and the specific index where data will be stored.
- **Lines 14–20:** Defines `lambda_handler`, the main function triggered by AWS Lambda.
    - **Lines 15–19:** Loops through each message record received from an SQS queue.
    - **Line 16:** Parses the JSON body of the SQS message into a Python dictionary (`payload`).
    - **Line 17:** Constructs a unique `document_id` for the OpenSearch document.
    - **Line 18:** Forms the complete OpenSearch API endpoint for indexing the document.
    - **Line 19:** Sends an HTTP `PUT` request to OpenSearch, securely authenticated, to index the `payload` as a document.
    - **Line 20:** Prints the HTTP response status and body for debugging purposes.

Querying data from OpenSearch

Once the data is indexed, we can search it using OpenSearch's Query DSL. Suppose we want to retrieve all temperature readings from devices located in the warehouse, we'd use the following query:

Ask

main.sh Shell

```
GET <opensearch-domain>/iot-devices/_search
{
  "query": {
    "match": {
      "location": "warehouse"
    }
  }
}
```

Query to get data of all devices in warehouse

This match query looks for all documents in the `iot-devices` index where the `location` field matches `warehouse`.

OpenSearch can also compute metrics using aggregations. To calculate the average temperature across all sensors, we can use the following query:

main.sh Shell

```
GET <opensearch-domain>/iot-devices/_search
{
  "size": 0,
  "aggs": {
    "avg_temp": {
      "avg": {
        "field": "temperature"
      }
    }
  }
}
```

Ask

Aggregation query to get average temperature

Here, we set `"size": 0` to skip returning documents and focus solely on the aggregation result, which computes the average temperature.

These indexing and query operations are fundamental building blocks in OpenSearch. They allow developers to create sophisticated search and analytics capabilities without managing complex infrastructure. Whether we're powering a search box, monitoring applications, or performing trend analysis, understanding indexing and query strategies is essential for building efficient, real-time AWS-native solutions.

Data source integration

Amazon OpenSearch can ingest data from a wide variety of AWS sources to support both real-time and batch analytics. Choosing the right integration depends on the application's data flow and latency needs.

- **Real-time ingestion:** For time-sensitive workloads like IoT telemetry or application monitoring, DynamoDB Streams combined with AWS Lambda is a common pattern. When a device writes to DynamoDB, a stream triggers a Lambda function that transforms and forwards the data into OpenSearch. Alternatively, *Amazon Kinesis Data Firehose* offers a fully managed pipeline that batches and delivers high-volume data directly into OpenSearch, which is ideal for log analytics and user activity streams.

Application fetching data from IoT devices

- **Batch ingestion:** For archived or less time-sensitive data, Amazon S often the source. We can schedule AWS Glue jobs or Lambda functions to periodically process and load this data into OpenSearch.

Ask

These integration options make it easy to stream, buffer, or schedule data flows into OpenSearch for analytics, monitoring, or search-driven applications.

Security and encryption

Amazon OpenSearch integrates with various AWS security services to protect the data throughout its lifecycle, from ingestion and indexing to querying and storage. We can integrate OpenSearch with IAM for limited access, KMS to encrypt data, and VPC for limited private network access.

- **IAM integration:** We can use AWS IAM to define fine-grained permissions that specify which users, roles, or services (like Lambda functions) are allowed to read from or write to specific indexes.
  For more granular control, OpenSearch supports fine-grained access control at the index, document, and field levels. This means developers can build applications where different users or groups see only the data they're authorized to access.
  Here's an example IAM policy that grants read-only access to a specific OpenSearch index, and also demonstrates how to allow write access to another:

index.json JavaScript (JSX)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<YOUR_ACCOUNT_ID>:user/<YOUR_IAM_USERNAMI
      },
      "Action": [
        "es:ESHttpGet",
        "es:ESHttpHead"
      ],
      "Resource": "arn:aws:es:<YOUR_REGION>:<YOUR_ACCOUNT_ID>:domain/
```

Ask

Restrictive policy for OpenSearch

- Here's a brief explanation of the policy provided above:
  - **Lines 4–14:** Grants read-only access (`es:ESHttpGet` and `es:ESHttpHead`) to a specific OpenSearch index. This allows the IAM user to perform search and metadata retrieval actions on the read index.
  - **Lines 15–26:** Grants write permissions (`es:ESHttpPut`, `es:ESHttpPost`, and `es:ESHttpDelete`) to a separate index. The IAM user is allowed to insert, update, and delete documents in this write-designated index.
  - **Lines 27–36:** Grants patch access (`es:ESHttpPatch`) across all indices and documents in the domain. This enables the IAM user to perform partial document updates without reindexing entire documents.

- **Encryption:** We can apply encryption at rest using AWS KMS and TLS (HTTPS) for data in transit. We can enforce HTTPS-only access to safeguard data integrity. These features protect data from unauthorized access both while stored and during transmission.
- **Private access with VPC:** We can also launch OpenSearch domains inside a VPC, ensuring that the service is not accessible over the public internet. This provides an extra layer of isolation and is recommended for workloads handling sensitive or regulated data.

By combining IAM policies, fine-grained access controls, encryption, and private networking, OpenSearch helps us enforce the principle of least privilege while keeping our data secure at every step.

Ask

Scaling and optimization

As the data grows, so does the complexity of managing performance and cost. We can use the following in OpenSearch:

- **Optimize shard count:** Tune shard configurations based on expected data size and access patterns. However, oversharding small datasets can consume excessive resources and overwhelm the cluster, leading to performance bottlenecks.
- **Implement Index State Management (ISM):** Automate the lifecycle management of indices to automatically move old data to warm or cold storage, or delete it. This also helps to save costs and manage storage efficiently.
- **Identify and address "hot shards":** Monitor for shards that receive disproportionate amounts of write traffic. Redistribute the data effectively by using balanced partition keys or routing strategies to prevent performance bottlenecks.
- **Leverage OpenSearch Serverless:** Utilize OpenSearch Serverless to automatically scale compute capacity based on usage. This solution is ideal for unpredictable or spiky workloads, as it removes the need to manually size and manage infrastructure.

OpenSearch gives us flexibility to scale smartly, whether we're planning for steady growth or sudden spikes.

Ask

Troubleshooting

Even in a managed environment, performance issues can surface in OpenSearch. Common problems include mapping errors, indexing failures, and slow queries. OpenSearch offers tools like slow logs and CloudWatch metrics to help pinpoint these issues.

By using these built-in observability features, we can detect and resolve indexing and search issues before they impact users.

Conclusion

Amazon OpenSearch is more than a search engine, it's a full stack, scalable solution for powering search and analytics across modern applications. Whether we're building a real-time monitoring platform, an interactive search experience, or a deep analytics dashboard, OpenSearch offers the indexing, querying, visualization, and observability tools to deliver high performance at scale. With tight integration into AWS services, built-in security, and flexible deployment options, it's a must-know tool for any developer aiming to build insight-driven

Ask